Test 1

```
python
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import joblib

# Step 1: Load the filtered dataset
df = pd.read_csv('Filtered_Pipe3D.csv')
print(f"Dataset loaded with {len(df)} rows and {len(df.columns)} columns.")

# Step 2: Define the target variable and features
# Assuming 'log_SFR_Ha' is the target variable; adjust if different
X = df.drop(columns=['log_SFR_Ha'])  # Features (all columns except the target)
y = df['log_SFR_Ha']                 # Target (log_SFR_Ha)

# Step 3: Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Training set: {X_train.shape[0]} rows, {X_train.shape[1]} columns")
print(f"Testing set: {X_test.shape[0]} rows, {X_test.shape[1]} columns")

# Step 4: Initialize and train the Random Forest model with default settings
rf_model = RandomForestRegressor(random_state=42)
rf_model.fit(X_train, y_train)

# Step 5: Make predictions on the testing set
y_pred = rf_model.predict(X_test)

# Step 6: Evaluate the model's performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"R-squared: {r2:.4f}")

# Optional Step 7: Hyperparameter tuning using GridSearchCV
# Define a grid of hyperparameters to search
param_grid = {
    'n_estimators': [100, 200, 300],      # Number of trees
    'max_depth': [10, 20, None],          # Maximum depth of trees
    'min_samples_split': [2, 5, 10]       # Minimum samples required to split a node
}

# Initialize GridSearchCV with 5-fold cross-validation
grid_search = GridSearchCV(estimator=RandomForestRegressor(random_state=42),
                           param_grid=param_grid,
                           cv=5,
```

```
                scoring='neg_mean_squared_error',
                n_jobs=-1)  # Use all available cores

# Fit the grid search to the training data
grid_search.fit(X_train, y_train)

# Get the best model from the grid search
best_rf_model = grid_search.best_estimator_

# Evaluate the tuned model on the testing set
y_pred_tuned = best_rf_model.predict(X_test)
mse_tuned = mean_squared_error(y_test, y_pred_tuned)
r2_tuned = r2_score(y_test, y_pred_tuned)
print(f"Tuned Mean Squared Error (MSE): {mse_tuned:.4f}")
print(f"Tuned R-squared: {r2_tuned:.4f}")

# Optional Step 8: Feature Importance Analysis
# Get feature importances from the best model
importances = best_rf_model.feature_importances_
feature_names = X_train.columns
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)
print("\nFeature Importances (Top 5):")
print(feature_importance_df.head())

# Save the feature importance dataframe to a CSV for further analysis
feature_importance_df.to_csv('feature_importances.csv', index=False)
print("Feature importances saved to 'feature_importances.csv'")

# Optional Step 9: Save the best model for future use
joblib.dump(best_rf_model, 'best_rf_model.pkl')
print("Best model saved to 'best_rf_model.pkl'")
```

Explanation of the Script

1. **Data Loading**
    - Loads the dataset from "Filtered_Pipe3D.csv" and prints its dimensions for verification.
2. **Target and Features**
    - Defines the target variable (assumed to be `'log_SFR_Ha'`) and features (all other columns).
    - Adjust the target column name if it differs in your dataset.
3. **Data Splitting**
    - Splits the data into 80% training and 20% testing sets with a fixed random seed for reproducibility.
4. **Model Training**

    - Initializes a Random Forest Regressor with default parameters and trains it on the training data.
5. **Prediction and Evaluation**
    - Makes predictions on the test set and evaluates performance using Mean Squared Error (MSE) and R-squared.
6. **Hyperparameter Tuning**
    - Uses `GridSearchCV` to optimize the Random Forest model by testing different combinations of `n_estimators`, `max_depth`, and `min_samples_split`.
7. **Tuned Model Evaluation**
    - Evaluates the tuned model on the test set and reports updated MSE and R-squared values.
8. **Feature Importance Analysis**
    - Extracts and sorts feature importances from the tuned model, displaying the top 5 most influential features.
    - Saves the full feature importance list to a CSV file.
9. **Model Saving**
    - Saves the best model to a file (`best_rf_model.pkl`) for future use.

Customization Notes

- **Target Variable**: If your SFR column is named differently (e.g., `'SFR_PETRORAD_R'`), replace `'log_SFR_Ha'` with the correct name in Step 2.
- **Hyperparameters**: Modify the `param_grid` in Step 7 to explore different ranges or add other parameters like `min_samples_leaf` based on your needs.
- **Feature Engineering**: Add preprocessing steps (e.g., scaling or creating new features) before Step 3 if required.
- **Dataset Path**: Ensure "Filtered_Pipe3D.csv" is in your working directory, or update the file path accordingly.

This script provides a complete workflow for building, refining, and analyzing a Random Forest model for your dataset.

Results

Dataset loaded with 10081 rows and 16 columns.
Training set: 8064 rows, 15 columns
Testing set: 2017 rows, 15 columns
Mean Squared Error (MSE): 0.2721
R-squared: 0.8231

Breakdown of the Results

Dataset Details

- **Total Dataset**:
  - Loaded with **10,081 rows** and **16 columns**.
  - This likely includes 15 feature columns (predictors) and 1 target column (e.g., star formation rate, SFR).
- **Training Set**:
  - **8,064 rows** and **15 columns** (features only, after splitting).
  - Used to train the Random Forest model.
- **Testing Set**:
  - **2,017 rows** and **15 columns** (features only).
  - Used to evaluate the model's performance on unseen data.

Model Performance Metrics

- **Mean Squared Error (MSE)**: **0.2721**
  - This measures the average squared difference between the predicted and actual SFR values.
- **R-squared (R^2)**: **0.8231**
  - This indicates the proportion of variance in the SFR that the model explains.

Interpretation of the Metrics

Mean Squared Error (MSE): 0.2721

- **What it means**:

    - The MSE represents the average error in the model's predictions. A value of 0.2721 suggests that, on average, the squared difference between predicted and actual SFR values is relatively small.
    - Since SFR is often expressed in logarithmic units (e.g., `log_SFR_Ha`), an MSE of 0.2721 indicates that predictions are reasonably accurate. For example, if your SFR values range from -2 to 2 (a typical log scale), this error is modest and suggests good predictive performance.
- **Context**:
    - Lower MSE values are better, and 0.2721 is a solid result, though its exact "goodness" depends on the scale and range of your SFR data.

R-squared (R^2): 0.8231

- **What it means**:
    - An R^2 of 0.8231 means that **82.31%** of the variance in the SFR is explained by the model. This is a strong indicator that the Random Forest is capturing most of the patterns in your data.
    - R^2 ranges from 0 to 1, where 1 is a perfect fit. A value above 0.8 is generally considered excellent, especially for complex datasets like those in astrophysics, where noise and variability are common.
- **Context**:
    - This high R^2 shows that the model generalizes well to the testing set, meaning it's not just memorizing the training data but learning meaningful relationships.

Why These Results Are Good

- **Strong Predictive Accuracy**:
    - The low MSE (0.2721) and high R^2 (0.8231) together suggest that your Random Forest model is effectively predicting SFR based on the 15 features.
- **Generalization**:
    - These metrics are calculated on the testing set (2,017 rows), which the model didn't see during training. This confirms that the model performs well on new data.
- **Astrophysical Relevance**:
    - For a task like predicting SFR, where factors like stellar mass, redshift, or gas content might be involved, an R^2 of 0.8231 indicates that the model is capturing significant physical relationships.

Suggestions for Improvement

While your model is already performing well, here are some optional steps to refine it or gain deeper insights:

1. **Tune Hyperparameters**:
    - The current model likely uses default settings. Adjusting parameters like the number of trees (`n_estimators`) or tree depth (`max_depth`) could lower the MSE further. Use tools like `GridSearchCV` in Python to find the best settings.
2. **Check Feature Importance**:
    - Identify which of the 15 features (e.g., stellar mass, redshift) are most critical to predictions. This can help you simplify the model or focus on key variables.
3. **Use Cross-Validation**:
    - Instead of a single train-test split, try k-fold cross-validation (e.g., 5-fold) to get a more reliable estimate of performance across different data subsets.
4. **Analyze Errors**:
    - Look at the residuals (predicted SFR - actual SFR) to see if the model struggles with specific cases (e.g., very high or low SFR galaxies). Plotting residuals can reveal patterns.
5. **Try Other Models**:
    - Compare the Random Forest to alternatives like Gradient Boosting or XGBoost to see if they perform better.

Conclusion

Your Random Forest model's test results are impressive: an MSE of 0.2721 and an R^2 of 0.8231 indicate strong predictive power and a good fit to the data. The model is successfully using the 15 features to predict SFR, explaining over 82% of its variability. If you're happy with this performance, you can use the model for further astrophysical analysis. If you want even better results, consider the improvement steps above. Let me know if you need help with any of these!

Validation Through K Fold Testing: Test 2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import KFold, cross_val_score, GridSearchCV, train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import joblib

# Step 1: Load the dataset
df = pd.read_csv('Filtered_Pipe3D.csv')  # Replace with your dataset file
print(f"Dataset loaded with {len(df)} rows and {len(df.columns)} columns.")

# Step 2: Define features (X) and target (y)
X = df.drop(columns=['log_SFR_Ha'])  # Replace 'log_SFR_Ha' with your target column
y = df['log_SFR_Ha']

# Step 3: Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Training set: {X_train.shape[0]} rows, Testing set: {X_test.shape[0]} rows")

# Step 4: Set up 5-fold cross-validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Step 5: Initialize the model (Random Forest Regressor)
model = RandomForestRegressor(random_state=42)

# Step 6: Perform cross-validation and calculate MSE for each fold
mse_scores = cross_val_score(model, X_train, y_train, cv=kf, scoring='neg_mean_squared_error')
mse_scores = -mse_scores  # Convert to positive MSE
print(f"MSE for each fold: {mse_scores}")
print(f"Average MSE from cross-validation: {np.mean(mse_scores):.4f}")

# Step 7: Define a grid of hyperparameters for tuning
param_grid = {
   'n_estimators': [100, 200, 300],
   'max_depth': [10, 20, None],
   'min_samples_split': [2, 5, 10]
}

# Step 8: Perform hyperparameter tuning with GridSearchCV
grid_search = GridSearchCV(estimator=RandomForestRegressor(random_state=42),
                           param_grid=param_grid,
                           cv=5,
                           scoring='neg_mean_squared_error',
                           n_jobs=-1)
grid_search.fit(X_train, y_train)
```

```
# Get the best hyperparameters and model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_
print(f"Best hyperparameters: {best_params}")

# Step 9: Train the best model on the full training set
best_model.fit(X_train, y_train)

# Step 10: Make predictions on the test set
y_pred = best_model.predict(X_test)

# Step 11: Evaluate the tuned model on the test set
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Tuned Mean Squared Error (MSE): {mse:.4f}")
print(f"Tuned R-squared: {r2:.4f}")

# Step 12: Save the best model
joblib.dump(best_model, 'best_rf_model.pkl')
print("Best model saved to 'best_rf_model.pkl'")

# Step 13: Plot predicted vs. actual values
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], 'r--')  # 45-degree line
plt.xlabel('Actual SFR')
plt.ylabel('Predicted SFR')
plt.title('Predicted vs. Actual Star Formation Rates')
plt.savefig('predicted_vs_actual.png')
plt.show()

# Step 14: Plot a histogram of residuals
residuals = y_test - y_pred
plt.figure(figsize=(8, 6))
plt.hist(residuals, bins=30, edgecolor='black')
plt.xlabel('Residuals (Actual - Predicted)')
plt.ylabel('Frequency')
plt.title('Histogram of Residuals')
plt.savefig('residuals_histogram.png')
plt.show()
```

Results

Dataset loaded with 10081 rows and 16 columns. Training set: 8064 rows, Testing set: 2017 rows MSE for each fold: [0.25457142 0.27341627 0.27918997 0.28171337 0.24007839] Average MSE from cross-validation: 0.2658 Best hyperparameters: {'max_depth': 20, 'min_samples_split': 5, 'n_estimators': 300} Tuned Mean Squared Error (MSE): 0.2743 Tuned R-squared: 0.8217

Here's a detailed breakdown of the results from your 5-fold cross-validation test using a Random Forest model, based on the information provided:

Key Results

- **Dataset:**
  - **Total rows: 10,081**
  - **Total columns: 16 (15 features + 1 target)**
- **Data Split:**
  - **Training Set: 8,064 rows (80%)**
  - **Testing Set: 2,017 rows (20%)**
- **Cross-Validation MSE (5-fold):**
  - **Fold MSEs: [0.2546, 0.2734, 0.2792, 0.2817, 0.2401]**
  - **Average MSE: 0.2658**
- **Best Hyperparameters:**
  - **`max_depth`: 20**
  - **`min_samples_split`: 5**
  - **`n_estimators`: 300**
- **Test Set Performance:**
  - **Tuned MSE: 0.2743**
  - **Tuned R-squared: 0.8217**

Interpretation

1. Dataset and Split

- **The dataset has 10,081 rows and 16 columns, split into 8,064 training rows (80%) and 2,017 testing rows (20%). This 80-20 split is a common practice, providing enough data for training while reserving a substantial portion for testing the model's performance on unseen data.**

2. Cross-Validation Performance

- **Mean Squared Error (MSE) per Fold:**
  - **Fold 1: 0.2546**
  - **Fold 2: 0.2734**
  - **Fold 3: 0.2792**
  - **Fold 4: 0.2817**
  - **Fold 5: 0.2401**
- **Average MSE: 0.2658**
- **What this means:**
  - **MSE measures the average squared difference between predicted and actual values—lower is better. The fold MSEs range from 0.2401 to 0.2817, showing consistent performance across different subsets of the training data. This stability suggests the model isn't overly sensitive to specific data splits and generalizes well within the training set.**

3. Hyperparameter Tuning

- **Best Hyperparameters:**
  - `max_depth=20`**: Limits tree depth to 20 levels, allowing the model to capture complex patterns without growing too unrestricted.**
  - `min_samples_split=5`**: Requires at least 5 samples to split a node, helping prevent overfitting by avoiding overly specific splits.**
  - `n_estimators=300`**: Uses 300 trees in the forest, creating a robust ensemble to reduce variance.**
- **What this means:**
  - **These settings strike a balance between model complexity (capturing patterns) and generalization (avoiding overfitting). The tuning process has optimized these parameters to improve performance on unseen data.**

4. Test Set Performance

- **Tuned MSE: 0.2743**
- **Tuned R-squared: 0.8217**
- **What this means:**
  - **The test MSE (0.2743) is slightly higher than the cross-validation average (0.2658), which is normal since the test set is unseen data. The small gap indicates good generalization.**
  - **R-squared (0.8217) shows that the model explains about 82.17% of the variance in the target variable. An R^2 above 0.8 is typically considered**

strong, meaning your model captures most of the data's variability and makes accurate predictions.

Overall Assessment

- **Strong Performance: The model achieves a low MSE (0.2743) and a high R-squared (0.8217) on the test set, indicating it predicts the target variable accurately and explains most of its variability.**
- **Good Generalization: The close alignment between the cross-validation MSE (0.2658) and test MSE (0.2743) shows the model performs well on unseen data, with minimal overfitting.**
- **Successful Tuning: The selected hyperparameters enhance the model's ability to balance complexity and robustness, leading to a slight improvement over the cross-validation baseline.**
- **Ready for Use: The best model is saved as `best_rf_model.pkl`, making it available for predictions or further analysis without retraining.**

Next Steps (Optional)

If you want to dive deeper or improve the model further, consider these steps:

1. **Feature Importances:**
    - **Extract the importance scores from the Random Forest to see which features (e.g., from your 15 predictors) most influence the target variable.**
2. **Residual Analysis:**
    - **Check the saved plots (if available) or create new ones to examine residuals (actual - predicted values). Randomly scattered residuals indicate a good fit; patterns might suggest areas for improvement.**
3. **Model Comparison:**
    - **Test alternative models (e.g., Gradient Boosting or XGBoost) to see if they outperform this Random Forest.**
4. **Further Tuning:**
    - **Expand the hyperparameter search if you have the resources, potentially lowering the MSE further.**

Conclusion

The results of your 5-fold cross-validation test show a well-performing Random Forest model. With an MSE of 0.2743 and an R-squared of 0.8217 on the test set, it's both accurate and reliable for predicting your target variable. The consistent cross-validation scores and optimized hyperparameters confirm its robustness. You can confidently use the saved model (`best_rf_model.pkl`) for predictions or proceed with additional analysis to refine it further. Let me know if you need assistance with any next steps!
Below is a complete, clear, and concise Python script based on your provided code, enhanced with additional features such as feature importance extraction, residual analysis, and model comparison. The script trains a Random Forest model with cross-validation and hyperparameter tuning, evaluates its performance, and compares it with Gradient Boosting and XGBoost models. Each step is well-commented for clarity.

Key Features Examination

```
python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import KFold, cross_val_score, GridSearchCV, train_test_split
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score
import joblib
import xgboost as xgb

# Step 1: Load the dataset
# Replace 'filtered_Pipe3D.csv' with your actual dataset file path
df = pd.read_csv('filtered_Pipe3D.csv')
print(f"Dataset loaded with {len(df)} rows and {len(df.columns)} columns.")

# Step 2: Define features (X) and target (y)
# Replace 'log_SFR_Ha' with your actual target column name
X = df.drop(columns=['log_SFR_Ha'])  # Features
y = df['log_SFR_Ha']                 # Target

# Step 3: Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Training set: {X_train.shape[0]} rows, Testing set: {X_test.shape[0]} rows")

# Step 4: Set up 5-fold cross-validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

```
# Step 5: Initialize the Random Forest model
rf_model = RandomForestRegressor(random_state=42)

# Step 6: Perform cross-validation and calculate MSE for each fold
mse_scores = cross_val_score(rf_model, X_train, y_train, cv=kf,
scoring='neg_mean_squared_error')
mse_scores = -mse_scores  # Convert to positive MSE
print(f"MSE for each fold: {mse_scores}")
print(f"Average MSE from cross-validation: {np.mean(mse_scores):.4f}")

# Step 7: Define a grid of hyperparameters for tuning
param_grid = {
   'n_estimators': [100, 200, 300],
   'max_depth': [10, 20, None],
   'min_samples_split': [2, 5, 10]
}

# Step 8: Perform hyperparameter tuning with GridSearchCV
grid_search = GridSearchCV(estimator=RandomForestRegressor(random_state=42),
                           param_grid=param_grid,
                           cv=5,
                           scoring='neg_mean_squared_error',
                           n_jobs=-1)
grid_search.fit(X_train, y_train)

# Get the best hyperparameters and model
best_params = grid_search.best_params_
best_rf_model = grid_search.best_estimator_
print(f"Best hyperparameters: {best_params}")

# Step 9: Train the best Random Forest model on the full training set
best_rf_model.fit(X_train, y_train)

# Step 10: Make predictions on the test set
y_pred_rf = best_rf_model.predict(X_test)

# Step 11: Evaluate the tuned Random Forest model on the test set
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)
print(f"Random Forest Tuned Mean Squared Error (MSE): {mse_rf:.4f}")
print(f"Random Forest Tuned R-squared: {r2_rf:.4f}")

# Step 12: Save the best Random Forest model
joblib.dump(best_rf_model, 'best_rf_model.pkl')
print("Best Random Forest model saved to 'best_rf_model.pkl'")

# Step 13: Extract and Display Feature Importances
importances = best_rf_model.feature_importances_
feature_names = X_train.columns
```

```
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
feature_importance_df = feature_importance_df.sort_values(by='Importance', ascending=False)
print("\nTop 5 Most Important Features:")
print(feature_importance_df.head())

# Plot feature importances
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_df['Feature'], feature_importance_df['Importance'], color='skyblue')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.title('Feature Importances from Random Forest')
plt.gca().invert_yaxis()  # Most important feature at the top
plt.savefig('feature_importances.png')
plt.show()

# Step 14: Residual Analysis
residuals_rf = y_test - y_pred_rf

# Plot predicted vs. actual values
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred_rf, alpha=0.5)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], 'r--')  # 45-degree line
plt.xlabel('Actual SFR')
plt.ylabel('Predicted SFR')
plt.title('Predicted vs. Actual Star Formation Rates (Random Forest)')
plt.savefig('predicted_vs_actual_rf.png')
plt.show()

# Plot histogram of residuals
plt.figure(figsize=(8, 6))
plt.hist(residuals_rf, bins=30, edgecolor='black')
plt.xlabel('Residuals (Actual - Predicted)')
plt.ylabel('Frequency')
plt.title('Histogram of Residuals (Random Forest)')
plt.savefig('residuals_histogram_rf.png')
plt.show()

# Plot residuals vs. predicted values
plt.figure(figsize=(8, 6))
plt.scatter(y_pred_rf, residuals_rf, alpha=0.5)
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel('Predicted Values')
plt.ylabel('Residuals (Actual - Predicted)')
plt.title('Residuals vs. Predicted Values (Random Forest)')
plt.savefig('residuals_vs_predicted_rf.png')
plt.show()

# Step 15: Model Comparison - Gradient Boosting
gb_model = GradientBoostingRegressor(random_state=42)
```

```
gb_model.fit(X_train, y_train)
y_pred_gb = gb_model.predict(X_test)
mse_gb = mean_squared_error(y_test, y_pred_gb)
r2_gb = r2_score(y_test, y_pred_gb)
print(f"\nGradient Boosting MSE: {mse_gb:.4f}")
print(f"Gradient Boosting R-squared: {r2_gb:.4f}")

# Step 16: Model Comparison - XGBoost
xgb_model = xgb.XGBRegressor(random_state=42)
xgb_model.fit(X_train, y_train)
y_pred_xgb = xgb_model.predict(X_test)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)
print(f"\nXGBoost MSE: {mse_xgb:.4f}")
print(f"XGBoost R-squared: {r2_xgb:.4f}")

# Step 17: Summarize Model Performance
print("\nModel Performance Summary:")
print(f"Random Forest - MSE: {mse_rf:.4f}, R-squared: {r2_rf:.4f}")
print(f"Gradient Boosting - MSE: {mse_gb:.4f}, R-squared: {r2_gb:.4f}")
print(f"XGBoost - MSE: {mse_xgb:.4f}, R-squared: {r2_xgb:.4f}")
```

Key Features and Enhancements

1. **Core Functionality**
   - **Loads a dataset from `'filtered_Pipe3D.csv'` and prepares features (`X`) and target (`y`) using `'log_SFR_Ha'` as the target column.**
   - **Splits data into 80% training and 20% testing sets.**
   - **Performs 5-fold cross-validation and hyperparameter tuning for a Random Forest Regressor using `GridSearchCV`.**
   - **Evaluates the tuned model with Mean Squared Error (MSE) and R-squared metrics.**
   - **Saves the best model as `'best_rf_model.pkl'`.**
2. **Feature Importance**
   - **Extracts and displays the top 5 most important features influencing the target variable.**
   - **Visualizes feature importances in a horizontal bar plot, saved as `'feature_importances.png'`.**
3. **Residual Analysis**
   - **Generates three diagnostic plots:**
     - **Predicted vs. Actual Values (`predicted_vs_actual_rf.png`): Assesses how well predictions align with actual values.**

        - **Histogram of Residuals (`residuals_histogram_rf.png`): Shows the distribution of prediction errors.**
        - **Residuals vs. Predicted Values (`residuals_vs_predicted_rf.png`): Checks for patterns in errors, ideally showing random scatter around zero.**
4. **Model Comparison**
    - **Trains and evaluates two additional models: Gradient Boosting and XGBoost.**
    - **Compares their performance (MSE and R-squared) against the Random Forest model in a summary section.**
5. **Visualization**
    - **All plots are saved as PNG files and displayed for immediate inspection.**

Customization Instructions

- **Dataset: Replace `'filtered_Pipe3D.csv'` with the path to your dataset file.**
- **Target Column: If your target variable is not `'log_SFR_Ha'`, update the column name in Step 2.**
- **Hyperparameters: Adjust the `param_grid` dictionary in Step 7 to explore different Random Forest configurations.**
- **Additional Models: Extend the comparison section by adding other models (e.g., Linear Regression) or tuning their hyperparameters.**

Output

When you run this script, it will:

- **Print dataset size, cross-validation MSE scores, best hyperparameters, and performance metrics for all models.**
- **Display and save plots for feature importances and residual analysis.**
- **Provide a concise summary comparing Random Forest, Gradient Boosting, and XGBoost.**

Results

Successfully installed nvidia-nccl-cu12-2.26.2 xgboost-3.0.0 (myenv) root@BigsbyGaming:/home/pmqr7# python features.py Dataset loaded with 10081 rows and 16 columns. Training set: 8064 rows, Testing set: 2017 rows MSE for each fold: [0.25457142 0.27341627 0.27918997 0.28171337 0.24007839] Average MSE from cross-validation: 0.2658 Best hyperparameters: {'max_depth': 20, 'min_samples_split': 5, 'n_estimators': 300} Random Forest Tuned Mean Squared Error (MSE): 0.2743 Random Forest Tuned R-squared: 0.8217 Best Random Forest model saved to 'best_rf_model.pkl'
Top 5 Most Important Features: Feature Importance 2 log_Mass_gas 0.752480 11 nsa_mstar 0.048074 0 log_Mass 0.044848 13 V-band_SB_at_Re 0.026654 12 vel_sigma_Re 0.018016 /home/pmqr7/features.py:88: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown plt.show() /home/pmqr7/features.py:101: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown plt.show() /home/pmqr7/features.py:110: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown plt.show() /home/pmqr7/features.py:120: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown plt.show()
Gradient Boosting MSE: 0.2663 Gradient Boosting R-squared: 0.8269
XGBoost MSE: 0.2919 XGBoost R-squared: 0.8103
Model Performance Summary: Random Forest - MSE: 0.2743, R-squared: 0.8217 Gradient Boosting - MSE: 0.2663, R-squared: 0.8269 XGBoost - MSE: 0.2919, R-squared: 0.8103

Results Breakdown

Dataset Overview

- **Dataset Size:**
  - **Loaded with 10,081 rows and 16 columns (15 features + 1 target, likely `log_SFR_Ha`).**
- **Data Split:**
  - **Training Set: 8,064 rows (80%)**
  - **Testing Set: 2,017 rows (20%)**

Cross-Validation Performance

- **MSE for Each Fold:**
  - **Fold 1: 0.2546**
  - **Fold 2: 0.2734**
  - **Fold 3: 0.2792**
  - **Fold 4: 0.2817**
  - **Fold 5: 0.2401**
- **Average MSE from Cross-Validation: 0.2658**
  - **This average MSE across the 5 folds indicates consistent performance during training, with a relatively tight range (0.2401 to 0.2817), suggesting the model generalizes well within the training data.**

Random Forest Model Performance

- **Best Hyperparameters:**
  - **`max_depth`: 20**
  - **`min_samples_split`: 5**
  - **`n_estimators`: 300**
  - **These settings balance model complexity and generalization, as determined by `GridSearchCV`.**
- **Tuned Random Forest Metrics:**
  - **MSE: 0.2743**
  - **R-squared: 0.8217**
  - **The test set MSE (0.2743) is slightly higher than the cross-validation average (0.2658), which is expected for unseen data. The small difference indicates good generalization.**
  - **An R^2 of 0.8217 means the model explains 82.17% of the variance in the target variable, a strong result for a regression task.**

Feature Importances

- **Top 5 Most Important Features:**

	Feature	Importance
2	log_Mass_gas	0.752480
11	nsa_mstar	0.048074
0	log_Mass	0.044848
13	V-band_SB_at_Re	0.026654
● 12	vel_sigma_Re	0.018016

    - **Interpretation:**
        - `log_Mass_gas` **(0.7525): This feature dominates with a 75.25% importance score, indicating that the gas mass in the galaxy is by far the most critical predictor of star formation rate (SFR). This aligns with astrophysical principles, as gas is the primary fuel for star formation.**
        - `nsa_mstar` **(0.0481) and** `log_Mass` **(0.0448): These represent stellar mass measurements, contributing about 4.8% and 4.5% respectively. Their importance reflects the role of stellar mass in influencing SFR, though they're far less impactful than gas mass.**
        - `V-band_SB_at_Re` **(0.0267): Surface brightness in the V-band at the effective radius contributes 2.67%, suggesting that the distribution of light (and thus stellar density) plays a role.**
        - `vel_sigma_Re` **(0.0180): Velocity dispersion within the effective radius contributes 1.8%, indicating that dynamical properties have a smaller but notable effect on SFR.**
    - **Insight: The dominance of `log_Mass_gas` suggests that your model heavily relies on gas availability to predict SFR, which is physically meaningful. However, the remaining features collectively contribute less than 25%, so you might explore whether additional features (e.g., metallicity gradients) could enhance the model.**

Model Comparison

- **Random Forest:**
    - **MSE: 0.2743**
    - **R-squared: 0.8217**
- **Gradient Boosting:**
    - **MSE: 0.2663**
    - **R-squared: 0.8269**
- **XGBoost:**
    - **MSE: 0.2919**

    - **R-squared: 0.8103**
- **Interpretation:**
    - **Gradient Boosting outperforms Random Forest slightly, with a lower MSE (0.2663 vs. 0.2743) and a higher R^2 (0.8269 vs. 0.8217). This suggests Gradient Boosting captures the data's patterns a bit better.**
    - **XGBoost performs worse, with a higher MSE (0.2919) and lower R^2 (0.8103), indicating it's less effective for this dataset without further tuning.**
    - **Conclusion: Gradient Boosting is the best-performing model so far, though the improvement over Random Forest is modest (about 3% reduction in MSE).**

Warnings

- **The `UserWarning: FigureCanvasAgg is non-interactive` messages indicate that `plt.show()` failed to display plots because you're running in a non-interactive environment (e.g., a server). However, the plots should still be saved as PNG files (e.g., `feature_importances.png`, `residuals_histogram_rf.png`) due to `plt.savefig()` calls in the script.**

Residual Analysis (Based on Saved Plots)

Since the script generated residual plots but couldn't display them, you'll need to check the saved files:

- `predicted_vs_actual_rf.png`**: Shows predicted vs. actual SFR values. Ideally, points should cluster tightly along the red dashed line (y=x), indicating accurate predictions.**
- `residuals_histogram_rf.png`**: Displays the distribution of residuals (actual - predicted). A symmetric, bell-shaped histogram centered at zero suggests good model fit.**
- `residuals_vs_predicted_rf.png`**: Plots residuals against predicted values. Random scatter around the red line (y=0) is ideal; patterns (e.g., a curve or funnel shape) indicate potential issues like missed nonlinear relationships or heteroscedasticity.**

Action:

- **Open these PNG files to inspect the plots. If residuals show patterns, the model might benefit from additional features or a different algorithm. If they're random, the model is fitting well.**

Next Steps Based on Results

1. **Adopt Gradient Boosting as the Primary Model**
    - **Since Gradient Boosting slightly outperforms Random Forest (MSE: 0.2663 vs. 0.2743, R^2: 0.8269 vs. 0.8217), consider using it as your primary model.**
    - **Action: Save the Gradient Boosting model for future use:**
    - `python`

```
joblib.dump(gb_model, 'best_gb_model.pkl')
```

    - **print("Best Gradient Boosting model saved to 'best_gb_model.pkl'")**
    - **Further Tuning: Tune Gradient Boosting's hyperparameters (e.g., `learning_rate`, `max_depth`) to potentially lower the MSE further:**
    - `python`

```
param_grid_gb = {
   'n_estimators': [100, 200, 300],
   'learning_rate': [0.01, 0.05, 0.1],
   'max_depth': [3, 5, 7]
}
grid_search_gb = GridSearchCV(estimator=GradientBoostingRegressor(random_state=42),
                              param_grid=param_grid_gb, cv=5, scoring='neg_mean_squared_error',
n_jobs=-1)
grid_search_gb.fit(X_train, y_train)
```

    - **print(f"Best Gradient Boosting hyperparameters: {grid_search_gb.best_params_}")**
2. **Leverage Feature Importances**
    - **Observation: `log_Mass_gas` dominates with 75.25% importance, while other features contribute much less.**
    - **Action:**
        - **Focus on Gas Mass: Since `log_Mass_gas` is critical, ensure its data quality is high (e.g., check for measurement errors or missing values).**

        - **Add Related Features: Explore adding features related to gas properties (e.g., gas metallicity, gas density) if available, as they might complement `log_Mass_gas` and reduce reliance on a single predictor.**
        - **Feature Engineering: Create interaction terms involving `log_Mass_gas` (e.g., `log_Mass_gas * log_Mass`) to capture combined effects.**
3. **Inspect Residual Plots**
    - **Action: Open the saved residual plots (`residuals_histogram_rf.png`, `residuals_vs_predicted_rf.png`) to check for patterns:**
        - **If residuals are randomly scattered around zero, the model fits well.**
        - **If you see patterns (e.g., a curve, funnel shape, or clustering), consider:**
            - **Adding nonlinear features (e.g., squared terms like `log_Mass_gas^2`).**
            - **Using a more complex model (e.g., a tuned Gradient Boosting or a neural network).**
    - **Optional: If you can't view the plots due to the non-interactive environment, transfer the PNG files to a local machine for inspection.**
4. **Improve XGBoost Performance**
    - **Observation: XGBoost underperformed (MSE: 0.2919, R^2: 0.8103) compared to Random Forest and Gradient Boosting.**
    - **Action: Tune XGBoost's hyperparameters to improve its performance:**
    - **`python`**

```
param_grid_xgb = {
   'n_estimators': [100, 200, 300],
   'learning_rate': [0.01, 0.05, 0.1],
   'max_depth': [3, 5, 7]
}
grid_search_xgb = GridSearchCV(estimator=xgb.XGBRegressor(random_state=42),
                      param_grid=param_grid_xgb, cv=5, scoring='neg_mean_squared_error',
n_jobs=-1)
grid_search_xgb.fit(X_train, y_train)
best_xgb_model = grid_search_xgb.best_estimator_
y_pred_xgb_tuned = best_xgb_model.predict(X_test)
mse_xgb_tuned = mean_squared_error(y_test, y_pred_xgb_tuned)
r2_xgb_tuned = r2_score(y_test, y_pred_xgb_tuned)
print(f"Tuned XGBoost MSE: {mse_xgb_tuned:.4f}")
```

    - **print(f"Tuned XGBoost R-squared: {r2_xgb_tuned:.4f}")**

5. **Address Plotting Warnings**
    - **The `UserWarning: FigureCanvasAgg is non-interactive` messages indicate that `plt.show()` isn't working in your environment. Since the plots are already saved as PNG files, you can safely remove the `plt.show()` calls to suppress the warnings:**
    - `python`

```
# Remove plt.show() after each plt.savefig()
plt.savefig('feature_importances.png')
```

    - **# plt.show() # Comment out or remove**
    - **Alternatively, transfer the saved PNG files (e.g., `feature_importances.png`) to a local machine with a graphical interface to view them.**

Summary

- **Model Performance: Gradient Boosting is the best model so far (MSE: 0.2663, R^2: 0.8269), slightly outperforming Random Forest (MSE: 0.2743, R^2: 0.8217). XGBoost needs tuning to compete.**
- **Feature Importance: `log_Mass_gas` is the dominant predictor (75.25% importance), suggesting gas mass is the key driver of SFR in your model.**
- **Residual Analysis: Check the saved plots to ensure residuals are random; patterns may indicate areas for improvement.**
- **Next Steps: Adopt Gradient Boosting, tune its hyperparameters, explore additional gas-related features, and refine XGBoost to potentially improve performance further.**

Test 3

Below is a complete Python script tailored to your query. It incorporates residual analysis guidance, adopts Gradient Boosting as the primary model, tunes its hyperparameters, leverages feature importances, improves XGBoost performance, and addresses plotting warnings by removing `plt.show()` calls since you're working

in a non-interactive environment. The script assumes your dataset is named `'filtered_Pipe3D.csv'` and the target variable is `'log_SFR_Ha'`. Adjust these as needed for your specific dataset.

Complete Python Script

```
python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import KFold, cross_val_score, GridSearchCV, train_test_split
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score
import joblib
import xgboost as xgb

# Step 1: Load the dataset
df = pd.read_csv('filtered_Pipe3D.csv')  # Replace with your dataset file
print(f"Dataset loaded with {len(df)} rows and {len(df.columns)} columns.")

# Step 2: Feature Engineering - Add interaction term
# Interaction between log_Mass_gas and log_Mass to capture combined effects
if 'log_Mass_gas' in df.columns and 'log_Mass' in df.columns:
    df['log_Mass_gas_times_log_Mass'] = df['log_Mass_gas'] * df['log_Mass']
    print("Added interaction term: log_Mass_gas_times_log_Mass")

# Step 3: Define features (X) and target (y)
X = df.drop(columns=['log_SFR_Ha'])  # Replace 'log_SFR_Ha' with your target column
y = df['log_SFR_Ha']

# Step 4: Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Training set: {X_train.shape[0]} rows, Testing set: {X_test.shape[0]} rows")

# Step 5: Set up 5-fold cross-validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Step 6: Initialize and tune Gradient Boosting model (adopted as primary model)
param_grid_gb = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7]
}
grid_search_gb = GridSearchCV(estimator=GradientBoostingRegressor(random_state=42),
                              param_grid=param_grid_gb,
                              cv=5,
```

```
                    scoring='neg_mean_squared_error',
                    n_jobs=-1)
grid_search_gb.fit(X_train, y_train)
best_gb_model = grid_search_gb.best_estimator_
print(f"Best Gradient Boosting hyperparameters: {grid_search_gb.best_params_}")

# Step 7: Train the best Gradient Boosting model on the full training set
best_gb_model.fit(X_train, y_train)

# Step 8: Make predictions on the test set
y_pred_gb = best_gb_model.predict(X_test)

# Step 9: Evaluate the tuned Gradient Boosting model on the test set
mse_gb = mean_squared_error(y_test, y_pred_gb)
r2_gb = r2_score(y_test, y_pred_gb)
print(f"Gradient Boosting Tuned Mean Squared Error (MSE): {mse_gb:.4f}")
print(f"Gradient Boosting Tuned R-squared: {r2_gb:.4f}")

# Step 10: Save the best Gradient Boosting model
joblib.dump(best_gb_model, 'best_gb_model.pkl')
print("Best Gradient Boosting model saved to 'best_gb_model.pkl'")

# Step 11: Feature Importance for Gradient Boosting
importances_gb = best_gb_model.feature_importances_
feature_names = X_train.columns
feature_importance_df_gb = pd.DataFrame({'Feature': feature_names, 'Importance':
importances_gb})
feature_importance_df_gb = feature_importance_df_gb.sort_values(by='Importance',
ascending=False)
print("\nTop 5 Most Important Features (Gradient Boosting):")
print(feature_importance_df_gb.head())

# Plot feature importances
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_df_gb['Feature'], feature_importance_df_gb['Importance'],
color='lightgreen')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.title('Feature Importances from Gradient Boosting')
plt.gca().invert_yaxis()
plt.savefig('feature_importances_gb.png')  # Save plot as PNG
# plt.show() removed to avoid warnings in non-interactive environment

# Step 12: Residual Analysis for Gradient Boosting
residuals_gb = y_test - y_pred_gb

# Plot predicted vs. actual values
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred_gb, alpha=0.5)
```

```
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], 'r--')  # 45-degree line
plt.xlabel('Actual SFR')
plt.ylabel('Predicted SFR')
plt.title('Predicted vs. Actual Star Formation Rates (Gradient Boosting)')
plt.savefig('predicted_vs_actual_gb.png')  # Save plot as PNG

# Plot histogram of residuals
plt.figure(figsize=(8, 6))
plt.hist(residuals_gb, bins=30, edgecolor='black')
plt.xlabel('Residuals (Actual - Predicted)')
plt.ylabel('Frequency')
plt.title('Histogram of Residuals (Gradient Boosting)')
plt.savefig('residuals_histogram_gb.png')  # Save plot as PNG

# Plot residuals vs. predicted values
plt.figure(figsize=(8, 6))
plt.scatter(y_pred_gb, residuals_gb, alpha=0.5)
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel('Predicted Values')
plt.ylabel('Residuals (Actual - Predicted)')
plt.title('Residuals vs. Predicted Values (Gradient Boosting)')
plt.savefig('residuals_vs_predicted_gb.png')  # Save plot as PNG

# Step 13: Tune XGBoost for comparison
param_grid_xgb = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7]
}
grid_search_xgb = GridSearchCV(estimator=xgb.XGBRegressor(random_state=42),
                               param_grid=param_grid_xgb,
                               cv=5,
                               scoring='neg_mean_squared_error',
                               n_jobs=-1)
grid_search_xgb.fit(X_train, y_train)
best_xgb_model = grid_search_xgb.best_estimator_
y_pred_xgb = best_xgb_model.predict(X_test)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
r2_xgb = r2_score(y_test, y_pred_xgb)
print(f"\nTuned XGBoost MSE: {mse_xgb:.4f}")
print(f"Tuned XGBoost R-squared: {r2_xgb:.4f}")

# Step 14: Summarize Model Performance
print("\nModel Performance Summary:")
print(f"Gradient Boosting - MSE: {mse_gb:.4f}, R-squared: {r2_gb:.4f}")
print(f"XGBoost - MSE: {mse_xgb:.4f}, R-squared: {r2_xgb:.4f}")
```

How to Use This Script

1. **Run the Script: Execute it in your Python environment. It will:**
    - **Load your dataset.**
    - **Perform feature engineering (e.g., adding an interaction term).**
    - **Train and tune a Gradient Boosting model (adopted as the primary model).**
    - **Save the model as `'best_gb_model.pkl'`.**
    - **Generate and save residual and feature importance plots as PNG files.**
    - **Tune an XGBoost model for comparison.**
2. **Inspect Residual Plots: Since you're in a non-interactive environment, transfer the saved PNG files to a machine with a graphical interface:**
    - `predicted_vs_actual_gb.png`**: Check if points cluster tightly along the red dashed line (y=x). This indicates accurate predictions.**
    - `residuals_histogram_gb.png`**: Look for a symmetric, bell-shaped distribution centered at zero, suggesting a good model fit.**
    - `residuals_vs_predicted_gb.png`**: Ensure residuals scatter randomly around the red line (y=0). Patterns (e.g., curves or funnel shapes) suggest missed nonlinear relationships or heteroscedasticity.**
3. **Evaluate Performance: Review the printed MSE and R-squared values for Gradient Boosting and XGBoost to compare their performance.**
4. **Next Steps Based on Residuals:**
    - **If Residuals Are Random: The model fits well. You can proceed with Gradient Boosting as your primary model.**
    - **If Patterns Exist: Consider adding nonlinear features (e.g., `df['log_Mass_gas_squared'] = df['log_Mass_gas'] ** 2`) or trying a more complex model (e.g., a neural network).**
5. **Leverage Feature Importances: Check the top features in the console output or `'feature_importances_gb.png'`:**
    - **If `log_Mass_gas` dominates, ensure its data quality and explore related features (e.g., gas metallicity, density) if available.**

Key Features of the Script

- **Gradient Boosting as Primary Model: Tuned with `GridSearchCV` and saved for future use, based on its superior initial performance (MSE: 0.2663, R^2: 0.8269).**

- **Feature Engineering: Adds an interaction term (`log_Mass_gas_times_log_Mass`) to capture combined effects.**
- **Residual Analysis: Saves diagnostic plots to assess model fit.**
- **XGBoost Tuning: Improves XGBoost performance for comparison.**
- **Plotting Warnings Addressed: Removes `plt.show()` calls, saving plots as PNG files instead.**

Notes

- **File Transfers: Since you can't view plots directly, move the PNG files (e.g., `feature_importances_gb.png`, `residuals_histogram_gb.png`) to a local machine for inspection.**
- **Further Improvements: If residuals show patterns or performance isn't satisfactory, let me know, and I can adjust the script (e.g., add more features or models).**

Results

Dataset loaded with 10081 rows and 16 columns.
Added interaction term: log_Mass_gas_times_log_Mass
Training set: 8064 rows, Testing set: 2017 rows
Best Gradient Boosting hyperparameters: {'learning_rate': 0.05, 'max_depth': 5, 'n_estimators': 200}
Gradient Boosting Tuned Mean Squared Error (MSE): 0.2596
Gradient Boosting Tuned R-squared: 0.8312
Best Gradient Boosting model saved to 'best_gb_model.pkl'

Top 5 Most Important Features (Gradient Boosting):

	Feature	Importance
2	log_Mass_gas	0.794929
11	nsa_mstar	0.045525
15	log_Mass_gas_times_log_Mass	0.044415
13	V-band_SB_at_Re	0.021827
0	log_Mass	0.017641

Tuned XGBoost MSE: 0.2606
Tuned XGBoost R-squared: 0.8306

Model Performance Summary:
Gradient Boosting - MSE: 0.2596, R-squared: 0.8312
XGBoost - MSE: 0.2606, R-squared: 0.8306

Dataset and Feature Engineering

- **Dataset Size: The dataset was loaded with 10,081 rows and 16 columns, providing a substantial amount of data for training and evaluation.**
- **Train-Test Split: The data was split into a training set of 8,064 rows and a testing set of 2,017 rows, following an 80-20 split, which is standard for robust model validation.**
- **Feature Engineering: An interaction term, `log_Mass_gas_times_log_Mass`, was added to capture the combined effect of gas mass and stellar mass. This is a useful step to model potential multiplicative relationships between these features.**

Gradient Boosting Model

- **Best Hyperparameters:**
  - **`learning_rate`: 0.05**
  - **`max_depth`: 5**
  - **`n_estimators`: 200**
- **Performance Metrics:**
  - **Mean Squared Error (MSE): 0.2596**
  - **R-squared: 0.8312**
- **Interpretation:**
  - **The MSE of 0.2596 indicates a low average prediction error, suggesting the model predicts the target variable (likely star formation rate, SFR) accurately.**
  - **An R-squared of 0.8312 means the model explains 83.12% of the variance in the target variable, which is an excellent result for a regression task.**

- **Model Storage: The best Gradient Boosting model was saved as `'best_gb_model.pkl'`, making it ready for future use or deployment.**

Feature Importance (Gradient Boosting)

The top 5 most important features in the Gradient Boosting model are:

Rank	Feature	Importance
1	**`log_Mass_gas`**	**0.7949**
2	**`nsa_mstar`**	**0.0455**
3	**`log_Mass_gas_times_log_Mass`**	**0.0444**
4	**`V-band_SB_at_Re`**	**0.0218**
5	**`log_Mass`**	**0.0176**

- **Key Observations:**
  - `log_Mass_gas` **is the dominant feature, contributing 79.49% to the model's predictions. This suggests gas mass is the primary driver of the target variable (e.g., SFR), which aligns with astrophysical principles.**
  - `nsa_mstar` **(a measure of stellar mass) and the interaction term** `log_Mass_gas_times_log_Mass` **contribute modestly at 4.55% and 4.44%, respectively, indicating secondary but meaningful roles.**
  - `V-band_SB_at_Re` **(surface brightness) and** `log_Mass` **have smaller contributions at 2.18% and 1.76%, respectively.**

XGBoost Model

- **Performance Metrics:**
  - **MSE: 0.2606**
  - **R-squared: 0.8306**
- **Interpretation:**

- **The XGBoost model performs very similarly to Gradient Boosting, with a slightly higher MSE (0.2606 vs. 0.2596) and a marginally lower R-squared (0.8306 vs. 0.8312).**
- **The differences are minimal, indicating both models are highly effective for this dataset.**

Model Performance Summary

Model	MSE	R-squared
Gradient Boosting	**0.2596**	**0.8312**
XGBoost	**0.2606**	**0.8306**

- **Comparison:**
  - **Gradient Boosting has a slight edge with a lower MSE and higher R-squared, but the performance gap is negligible.**
  - **Both models achieve strong predictive power, explaining over 83% of the variance in the target variable.**

Key Takeaways

1. **Model Effectiveness:**
   - **Both Gradient Boosting and XGBoost deliver excellent performance, with Gradient Boosting slightly outperforming XGBoost. The saved Gradient Boosting model (`'best_gb_model.pkl'`) is a reliable choice for predictions.**
2. **Feature Insights:**
   - **The overwhelming importance of `log_Mass_gas` (79.49%) highlights its critical role in driving the target variable, likely SFR. The interaction term `log_Mass_gas_times_log_Mass` also adds value, supporting the feature engineering effort.**
3. **Next Steps:**

- **Validate Model Fit: If residual plots (e.g., predicted vs. actual, residuals histogram) were generated, inspect them to ensure the model captures the data's patterns without systematic errors.**
- **Enhance Features: Consider adding more features related to gas properties (e.g., metallicity, density) or additional interaction terms to further improve performance.**
- **Deployment: The saved Gradient Boosting model can be loaded and used for predictions on new data.**